



Technical Q&A Guide

Questions Rishitha will be asked — with clear, confident answers

[Tech Stack Questions](#)[Code Explanations](#)[Architecture Decisions](#)[Live Demo Tips](#)



Tech Stack Choices

VERY LIKELY TO BE ASKED

Why did you choose Spring Boot for the backend?

Say this: Spring Boot is the industry-standard Java framework for building REST APIs. It comes with built-in security (Spring Security), data handling (JPA/Hibernate), and removes most of the boilerplate configuration. Since this is a Java Full Stack project, Spring Boot was the natural choice.

It also has auto-configuration, meaning I didn't have to manually set up the server or database connection — Spring Boot does that automatically based on what's on the classpath.

HOW TO SAY IT CONFIDENTLY

"I used Spring Boot because it is the most widely used Java backend framework in the industry, especially for REST APIs. It handles authentication, database access, and security out of the box, which let me focus on the business logic."

LIKELY TO BE ASKED

Why Angular for the frontend? Why not React?

Say this: Angular is a complete, opinionated framework maintained by Google. It includes everything — routing, forms, HTTP client, and TypeScript by default. React is a library, not a full framework, so it requires piecing together many third-party tools.

For an enterprise application like a customer service dashboard, Angular's structure and built-in tools make it easier to maintain and scale. The standalone component model in Angular 17 also reduces unnecessary complexity.

HOW TO SAY IT CONFIDENTLY

"Angular is a complete framework, not just a library. For a structured, multi-role enterprise application, Angular's built-in routing, forms, and dependency injection are much more suitable than assembling separate libraries."

VERY LIKELY TO BE ASKED

Why JWT for authentication? What is JWT?

What JWT means: JSON Web Token. It's a small encoded string that proves who the user is. After login, the server generates a JWT and sends it to the browser. The browser stores it and sends it with every API request. The server verifies the token without needing to check a database for every request.

Why we used it: JWT is *stateless* — the server does not need to remember sessions. This makes it faster, more scalable, and perfect for REST APIs. It also carries the user's role inside it, so the backend can check permissions instantly.

```
// What a JWT looks like (three parts separated by dots):  
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbkBJc2QuY29tIiwicm9sZSI6IkFETU0In0.XxYz...  
  
// Part 1: Header (algorithm used)  
// Part 2: Payload (user email, role, expiry)  
// Part 3: Signature (proves it hasn't been tampered with)
```

HOW TO SAY IT CONFIDENTLY

"I used JWT because it is stateless — the server does not need to store session data. The token itself contains the user's identity and role, so every API request is self-contained and can be verified instantly."

MIGHT BE ASKED

Why H2 database? Why not MySQL or PostgreSQL?

H2 is an in-memory / file-based database that is embedded directly in the Java application. It requires zero installation and zero configuration — it just works.

For a capstone demo project, H2 is ideal because: (1) anyone can run it without installing a database server, (2) it auto-creates and auto-seeds tables on startup, and (3) switching to MySQL or PostgreSQL in production would require only a one-line change in the configuration file.

HOW TO SAY IT CONFIDENTLY

"I chose H2 for the demo because it is embedded — no separate database installation is required. For production, switching to MySQL would only require changing the database URL in application.properties."



Authentication & Security

VERY LIKELY TO BE ASKED

Explain how the login flow works from start to end.

Here is the complete flow when a user logs in:



For every subsequent request (like loading tickets), the JWT interceptor automatically adds the token to the request header: `Authorization: Bearer <token>`. Spring Security validates the token before allowing access.

HOW TO SAY IT CONFIDENTLY

"When the user logs in, the backend validates the credentials and returns a JWT token. Angular stores this token in sessionStorage and attaches it automatically to every API call through an HTTP interceptor. Spring Security validates the token on every request."

LIKELY TO BE ASKED

What is the JWT interceptor and why did you use it?

An interceptor in Angular intercepts every HTTP request before it is sent. Instead of manually adding the token to every API call, the interceptor does it automatically.

```
// File: frontend/src/app/interceptors/jwt.interceptor.ts
export const jwtInterceptor: HttpInterceptorFn = (req, next) => {
  const token = sessionStorage.getItem('token');
  if (token) {
    req = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });
  }
  return next(req);
}
```

```
return next(req);  
};
```

HOW TO SAY IT CONFIDENTLY

"The JWT interceptor automatically attaches the token to every HTTP request. Without it, I would have to manually add the Authorization header in every single service call — the interceptor keeps the code clean and DRY."

MIGHT BE ASKED

How does the auth guard work? What happens if someone tries to access the admin page directly?

The auth guard runs before Angular renders any protected route. It checks two things: (1) is there a valid token in sessionStorage, and (2) does the user's role match the required role for that route.

```
// File: frontend/src/app/guards/auth.guard.ts  
export const authGuard: CanActivateFn = (route) => {  
  const token = sessionStorage.getItem('token');  
  const user = JSON.parse(sessionStorage.getItem('user') || 'null');  
  
  if (!token || !user) {  
    router.navigate(['/login']); // not logged in → go to login  
    return false;  
  }  
  
  const requiredRole = route.data?.['role'];  
  if (requiredRole && user.role !== requiredRole) {  
    router.navigate(['/login']); // wrong role → go to login  
    return false;  
  }  
  return true;  
};
```

If someone types `/admin` in the URL bar without being logged in, the guard catches it and redirects them to the login page immediately.



Key Code Explanations

VERY LIKELY TO BE ASKED

Explain the DataInitializer — how does the demo data get created?

DataInitializer.java is a Spring component that runs automatically when the application starts up (via the `@PostConstruct` annotation). It checks if there are already users in the database, and if not, it creates all the demo data.

```
// File: src/main/java/com/wipro/csd/config/DataInitializer.java

@PostConstruct      // runs automatically after Spring starts
public void init() {
    if (userRepo.count() > 0) return; // skip if data already exists

    // Create users with encrypted passwords
    User admin = userRepo.save(User.builder()
        .name("System Admin")
        .email("admin@csd.com")
        .password(passwordEncoder.encode("admin123")) // password is HASHED
        .role(User.Role.ADMIN)
        .build());

    // Then creates managers, reps, customers, tickets, outages...
}
```

HOW TO SAY IT CONFIDENTLY

"DataInitializer uses the `@PostConstruct` annotation which tells Spring to run this method immediately after the application context loads. It creates all demo users, tickets, and outages only if the database is empty, preventing duplicate data on restarts."

VERY LIKELY TO BE ASKED

How does role-based access work — can a customer see admin data?

Role-based access is enforced at **two levels**:

Level 1 — Frontend (Angular Auth Guard): Routes are protected with `canActivate: [authGuard]`.

The guard checks the user's role from sessionStorage before rendering the page.

Level 2 — Backend (Spring Security): Every API endpoint is secured. Even if someone bypasses the UI, the backend rejects unauthorized requests.

```
// In app.routes.ts — admin route is protected
{
  path: 'admin',
  loadComponent: () => import('./pages/admin/admin.component'),
  canActivate: [authGuard],
  data: { role: 'ADMIN' } // only ADMIN role can access this
}
```

HOW TO SAY IT CONFIDENTLY

"Role-based access is enforced at two levels — the Angular auth guard blocks the wrong role from even seeing the page, and Spring Security independently verifies the JWT role on every API call. So even if someone manipulates the URL, the backend will reject the request."

MIGHT BE ASKED

What is lazy loading and why did you use it in Angular?

Lazy loading means Angular only downloads the code for a page when the user actually navigates to it — not all at once when the app first loads. This makes the initial load faster.

```
// Instead of importing AdminComponent directly, we use:
{
  path: 'admin',
  loadComponent: () =>
    import('./pages/admin/admin.component')
    .then(m => m.AdminComponent)
  // ^ code only downloaded when user goes to /admin
}
```

HOW TO SAY IT CONFIDENTLY

"Lazy loading splits the application into smaller chunks. The admin dashboard code is only downloaded if the user navigates to the admin page. This reduces the initial load time significantly."

MIGHT BE ASKED**How does the chatbot work?**

The chatbot is a rule-based keyword matching system. It has 12 predefined topic categories (internet, billing, TV, mobile, etc.). When the user types a message, the system converts it to lowercase and checks if any keyword from any category is present in the message. If a match is found, it returns the pre-written response for that category.

```
// File: frontend/src/app/shared/chatbot/chatbot.component.ts
function getBotResponse(input: string): string {
  const lower = input.toLowerCase();
  for (const entry of BOT_RESPONSES) {
    if (entry.k.some(k => lower.includes(k))) {
      return entry.r; // return matching response
    }
  }
  // no match → generic fallback response
  return `I understand you need help with "${input}". Please raise a ticket.`;
}
```

HOW TO SAY IT CONFIDENTLY

"The chatbot uses keyword matching — when a user types a message, it scans for known keywords like 'internet', 'billing', or 'slow' and returns the appropriate pre-defined response. If no keyword matches, it falls back to suggesting the user raise a support ticket."



Architecture Questions

VERY LIKELY TO BE ASKED

Explain the overall architecture of the project.

NexCare uses a **client-server architecture** with complete separation between frontend and backend:



The Angular frontend communicates with the Spring Boot backend exclusively through REST APIs. A proxy configuration in development (`proxy.conf.json`) forwards all `/api` requests from port 4200 to 8080, eliminating CORS issues during development.

Spring Security wraps every API endpoint, validating the JWT token on every request before allowing access to any data.

HOW TO SAY IT CONFIDENTLY

"The architecture is a standard three-tier design: presentation tier (Angular), application tier (Spring Boot REST APIs), and data tier (H2 database). These are completely decoupled — the frontend only knows the API contract, not the implementation details."

MIGHT BE ASKED

What is the proxy config and why is it needed?

By default, browsers block requests from one port (4200) to another port (8080) for security reasons — this is called the CORS (Cross-Origin Resource Sharing) policy.

The proxy config (`frontend/proxy.conf.json`) tells the Angular development server: "Any request starting with `/api` should be forwarded to port 8080." The browser only ever talks to port 4200, so it never triggers the CORS restriction.

```
// File: frontend/proxy.conf.json
{
  "/api": {
    "target": "http://localhost:8080", // forward to backend
```

```
"secure": false,  
"changeOrigin": true  
}  
}
```

HOW TO SAY IT CONFIDENTLY

"The proxy config solves the CORS problem by routing all API calls through the Angular dev server itself. The browser sees all traffic on port 4200, so no cross-origin request is made."

MIGHT BE ASKED

Why did you use standalone components in Angular instead of NgModules?

Angular 17 introduced standalone components as the new recommended approach. Traditional NgModules required declaring and importing every component in a module file, which added boilerplate code.

Standalone components declare their own imports directly, making them self-contained. This means each component explicitly lists what it needs (e.g., `CommonModule`, `FormsModule`) without relying on a shared module.

HOW TO SAY IT CONFIDENTLY

"Angular 17 recommends standalone components because they are self-contained and eliminate the need for NgModule declarations. Each component is independent and imports only what it needs, which makes the code cleaner and tree-shaking more effective."



Live Demo — "Show Me" Questions

VERY LIKELY DURING DEMO

Show me how a ticket gets created and how a representative resolves it.

Steps to demonstrate:

1. Log in as **Customer** (alice@gmail.com / Customer@123)
2. Click **My Tickets** in sidebar → click **" + Raise New Ticket"**
3. Select domain (e.g. Internet Services), enter subject and description → Submit

4. Log out → Log in as **Representative** (mike.rep@csd.com / Welcome@123)
5. Click **My Tickets** → find the new ticket with status OPEN
6. Click **Update** → set status to **In Progress** → save
7. Click **Update** again → set status to **Closed** → save
8. Log out → Log back in as Customer → ticket now shows as Resolved with a Rate button

VERY LIKELY DURING DEMO

Show me how the admin manages employees.

Steps to demonstrate:

1. Log in as **Admin** (admin@csd.com / admin123)
2. Click **Employees** in the sidebar
3. Click **" + Add Employee "** → fill in name, email, role = Representative, assign to a manager → Save
4. The new employee appears in the table
5. Click **Edit** on any employee to modify details
6. Click **Delete** → confirm to remove

MIGHT BE ASKED DURING DEMO

What happens if I try to access the admin URL directly without logging in?

Type `http://localhost:4200/#/admin` in the browser address bar while not logged in.

What happens: The Angular auth guard intercepts the navigation, sees there is no token in sessionStorage, and immediately redirects to the login page. The admin page never loads.

Even if you somehow got a token from a different role (e.g., customer token), the guard checks the role and would still redirect.